

Assignment 4: Advanced State Management and Persistence

Release Date: October 26, 2025

Due Date: Sunday, November 9, 2025, 11:59 PM EST

Weight: 12.5% of final grade

Objective

Extend your Fitness Tracker app from Assignment 3 by replacing the Context API with Redux Toolkit for centralized state management, and by adding persistent local storage using React Native Async Storage.

The app's UI and navigation (Expo Router) remain unchanged. This assignment focuses on modern, scalable global state management and data persistence in mobile applications.

Technologies

- **React Native:** For building the mobile app UI and functionality.
- **TypeScript:** For type-safe development.
- **Expo CLI:** For project setup, development, and testing via Expo Go or emulators (use Node.js v22).
- **Expo Router:** For file-based navigation with dynamic routes (reuse from Assignment 3).
- **Redux Toolkit:** For centralized, predictable state management
- **React Native Async Storage:** For persistent local data

Assignment Requirements

Build upon your completed [Assignment 3 Fitness Tracker project](#). All screens (Home, Add/Edit, Details) and navigation must remain functional. Replace the Context API with Redux Toolkit, and add Async Storage so activity data is saved between sessions.

1 Redux Toolkit Setup

- **Starter Code:** Begin with your completed Assignment 3 project as the foundation.
Download the reference starter project by [clicking this link](#) to obtain `assignment-4.tar.gz`.
This archive includes:
 - Updated `package.json` and `package-lock.json` (adding Redux Toolkit and Async Storage dependencies)

- A starter `app/_layout.tsx` configured for Redux integration
- A starter `store/store.ts` for setting up the Redux store
- A starter `features/activities/activitiesSlice.ts` defining the `activities` slice
- A starter `storage/activitiesStorage.ts` for handling persistent storage
- The same screen structure and components from Assignment 3

After downloading, extract the `.tar.gz` file, navigate to the project directory, and run:

```
npm install --legacy-peer-deps
```

to install all dependencies, and edit the necessary files to implement the required functionality.

You must update the screens inside the `app/` directory to replace all Context API logic with Redux Toolkit hooks. The UI and navigation remain unchanged.

- **Store and Slice Configuration:**

1. `store/store.ts`

- Configure and export the Redux store using `configureStore()` from `@reduxjs/toolkit`.
- Import the reducer from `features/activities/activitiesSlice` and include it under the key `activities`.
- Export `RootState` and `AppDispatch` types for use with `useSelector` and `useDispatch`.

2. `features/activities/activitiesSlice.ts`

- Define the `ActivitiesState` interface:

```
interface ActivitiesState {
  activities: Activity[];
  loaded: boolean;
}
```

Note

The `loaded` flag tracks whether activities have been initialized from storage (initially `false`). This follows Redux patterns for handling async data loading (e.g., preventing UI renders until data is ready). For this assignment, you only need to set it to `true` after successful load — no UI changes are required.

- Initialize the state with an empty activities array and `loaded: false`.
- Once activities have been successfully loaded from Async Storage, set `loaded` to `true` in the `setActivities` reducer.
- Use `createSlice()` to define reducers:
 - `addActivity(activity: Omit<Activity, "id">)`

- `updateActivity({ id, updated }: { id: string; updated: Omit<Activity, "id"> })`
- `deleteActivity(id: string)`
- `setActivities(Activity[])`

Note

In the `addActivity`, `updateActivity`, and `deleteActivity` reducers, call `saveActivities` after updates. This is a simple approach for this assignment to demonstrate persistence basics. In production apps, prefer [Redux thunks or middleware](#) for async side effects to keep reducers pure and handle loading/errors better — no need to implement that here.

- Export:

- Actions: `addActivity`, `updateActivity`, `deleteActivity`, `setActivities`
- Selectors: `selectActivities`, `selectActivityById`
- Default reducer

3. `app/_layout.tsx`

- Wrap the app in the Redux `<Provider>` so that all screens can access the global store.

- **Types:**

Reuse `Activity` from Assignment 3 (`types.ts`), and remove `ActivityContextType`.

2 Integrating Redux into Screens

You will modify the custom component `ActivityListItem` and existing screens under the `app/` folder to migrate from the Context API to Redux Toolkit.

Replace all previous `useActivities` (Context) calls with Redux hooks:

- **ActivityListItem Component (`components/ActivityListItem.tsx`)**
 - Replace `useActivities` with `useDispatch` from `"react-redux"`.
 - Dispatch `deleteActivity(activity.id)` in `handleDelete`.
 - Remove Context import; keep UI and navigation unchanged.
- **Home Screen (`app/index.tsx`)**
 - Use `useSelector(selectActivities)` to read all activities.
 - Use `useDispatch()` to dispatch `deleteActivity`.
 - Keep navigation, styling, and test IDs unchanged.
- **Add/Edit Screen (`app/add-edit.tsx`)**
 - Use `useSelector` to prefill data when editing.
 - On submit, dispatch `addActivity` or `updateActivity`.

- Navigate to Home using the Expo Router API.
- **Details Screen (`app/details/[id].tsx`)**
 - Use `useSelector(selectActivityById)` to display activity details.
 - Edit button navigates to `add-edit?id=...`.
 - Delete button dispatches `deleteActivity` and navigates to Home.

All test IDs (`edit-button` , `delete-button` , `activity-list` , etc.) must remain identical to Assignment 3 for autograding.

3 Persistence with Async Storage

Implement the async `loadActivities` and `saveActivities` functions in `storage/activitiesStorage.ts` for saving and loading activities.

- **Goal:** Ensure that users' activities persist across app restarts using React Native Async Storage.
- **Behavior:**
 - In `app/_layout.tsx` , use `useEffect` to call `loadActivities` on app startup, then dispatch `setActivities` to restore the state.
 - In `features/activities/activitiesSlice.ts` reducers (`addActivity` , `updateActivity` , `deleteActivity`), call `saveActivities` after updating state.
 - Use asynchronous functions and proper error handling (e.g., try/catch, return empty array on load failure) to ensure reliability. Use key `@FitnessTracker_activities` in `storage/activitiesStorage.ts` .

Testing Your Implementation

Functionality Checklist

- ☐ Redux store configured with one slice (`activitiesSlice`).
- ☐ All CRUD operations dispatch Redux actions correctly.
- ☐ Async Storage automatically loads saved activities on startup.
- ☐ Async Storage persists data after every add/edit/delete.
- ☐ Screens and navigation unchanged from Assignment 3.
- ☐ App runs in Expo Go or simulator/emulator without errors.

Note

The autograder tests Redux store setup, reducer logic, Async Storage persistence, and functional navigation flow.

It will not recheck UI styling, but structure and test IDs must match Assignment 3.

Input validation will not be rechecked as well, but please keep it in your implementation for completeness.

Manual Testing

1. Start your app locally:

```
npx expo start
```

2. Test using:

- **Expo Go:** Scan the QR code with the Expo Go app.
- **Emulator:** Press `i` (iOS simulator) or `a` (Android emulator).
- **Inspector:** Press `m` to open the developer menu.

3. You may check with the following flow:

1. Add several activities → restart the app → verify they remain visible.
2. Edit or delete activities → restart again → ensure updates persist.
3. Test navigation: Home → Details → Edit → Home.

Sample Test Cases

Download `a4.sample.test.js` [here](#) and save it to your project directory. Run:

```
npm test
```

Review output to fix failing tests. The sample tests cover 70% of the autograder's criteria.

Submission Instructions

1 Generate the Archive

From the parent directory of your project, create a `.tar.gz` archive with:

```
cd /path/to/parent/directory
tar zcvf 1234567890-a4.tar.gz assignment-4
```

- Replace `assignment-4` with the name of your project directory.
- Replace `1234567890` with your student number.
- The archive must contain **exactly one top-level folder** (your project directory) with the required files.
- **Do not** include `node_modules/`, `.expo/`, or other generated files.

Your project directory (e.g., `assignment-4`) should **exactly** match:

```
└─ assignment-4
  └─ app
    ├── _layout.tsx
    ├── index.tsx
    ├── add-edit.tsx
    └─ details
      └─ [id].tsx
  └─ app.json
  └─ assets
  └─ components
    ├── ActionButton.tsx
    ├── ActivityListItem.tsx
    ├── Card.tsx
    ├── DetailsCard.tsx
    └── PrimaryButton.tsx
  └─ constants
    └─ colors.ts
  └─ features
    └─ activities
      └─ activitiesSlice.ts
  └─ storage
    └─ activitiesStorage.ts
  └─ store
    └─ store.ts
  └─ styles
    └─ globalStyles.ts
  └─ package-lock.json
  └─ package.json
  └─ tsconfig.json
  └─ types.ts
```

⚠ Any submission that fails to meet the specified format or structure and requires TA intervention for the autograder to work will receive a **20-point deduction**.

2 Verify Submission Structure

To avoid a 20-point deduction, use the provided Python script to check your `.tar.gz` file before submission.

1. [Download `check\_submission\_structure.py`](#) and save it in the parent directory of your project.
2. Ensure you have Python installed. Check by running `python --version` in your terminal/command prompt. If not installed, download it from python.org.
3. From the parent directory of your project, run:

```
python check_submission_structure.py 1234567890-a4.tar.gz
```

- Replace `1234567890` with your student number.

3 Submit to Quercus

Submit your `.tar.gz` file to [Quercus](#) [↗].

Note

You are allowed to submit unlimited times. Only your latest submission before the deadline will be graded. Quercus automatically appends a suffix to the file name after the first submission — this will not affect grading.

Grading Scheme (100 Points)

The autograder runs the app to test Redux Toolkit state updates, AsyncStorage persistence, and navigation. Partial marks are awarded for partial functionality. UI appearance and input validation are not graded.

Add Activity (30 Points)

Add `Running, 20 min` → calories auto-calculated → then add `Walking, 30 min, 211 cal`.

Home List Display (15 pts)

- Running shows correct info:
 - Type (3 pts), Duration (3 pts), Calories (3 pts)
- Walking shows correct info:
 - Type (2 pts), Duration (2 pts), Calories (2 pts)

AsyncStorage Persistence (15 pts)

- Stored under key `@FitnessTracker_activities` (3 pts)
- Running stored with correct fields: type (3 pts), duration (3 pts), calories (3 pts)
- Walking entry also included correctly (3 pts)

Load After Restart (18 Points)

On app remount, previous activities should reappear.

- Running restored correctly: type (3 pts), duration (3 pts), calories (3 pts)
- Walking restored correctly: type (2 pts), duration (2 pts), calories (2 pts)

- Slice sets `loaded = true` (3 pts)

Update Activity (32 Points)

Edit the first activity from Running → Swimming.

Navigation + Details (8 pts)

- Tapping item navigates to Details (2 pts)
- Details shows correct Running info (2 pts + 2 pts + 2 pts)

Edit Screen Prefill (6 pts)

- Type (2 pts), Duration (2 pts), Calories (2 pts)

After Updating (8 pts)

- Returns to Home (2 pts)
- Home shows updated **Type: Swimming** (4 pts)
- Duration + calories unchanged (2 pts each)

Persistence (10 pts)

- Same `id` preserved (4 pts)
- Updated type (4 pts)
- Duration + calories unchanged (2 pts each)

Delete Activity (20 Points)

Delete activities from Details.

Delete 1st Activity (14 pts)

- Returns to Home (2 pts)
- Removed from list (3 pts)
- Remaining item still shown (3 pts)
- AsyncStorage updated: removed item absent (3 pts), remaining item present (3 pts)

Delete Last Activity (6 pts)

- Home list no longer shows it (3 pts)
- Stored JSON array is empty (3 pts)

Resources

- **Course Materials:**

- [Lecture 6: State Management in React Native](#)
- [Lecture 7: Data Persistence](#)
- [Lecture 8: Handling Async Side Effects in Redux](#)
- **External References:**
 - [Redux Toolkit Documentation](#) ↗
 - [React Native Async Storage Documentation](#) ↗

Questions?

1. Discussion Board:

- Post questions on [course discussion board](#) ↗
- Search existing discussions first
- Use clear titles and provide relevant code snippets

2. Office Hours:

- Time: Mondays, 2:00 PM — 3:00 PM
- Location: Room 7206, Bahen Centre for Information Technology

3. Tips for Getting Help:

- Start early to allow time for questions
- Be specific about your problem
- Share what you've tried
- Include relevant error messages

Last updated on September 1, 2025